

```
1 ### md
2 # Ejercicio final de la semana 1
3
4 Para demostrar que estás familiarizado con la API de
  OpenAI y también con Ollama, crea una herramienta que
  responda a una pregunta técnica
5 y la explique. ¡Esta es una herramienta que podrás
  usar durante el curso!
6 ###
7 # imports
8
9 import os
10 import requests
11 import json
12 from typing import List
13 from dotenv import load_dotenv
14 from IPython.display import Markdown, display,
  update_display
15 from openai import OpenAI
16
17 ###
18 # constantes
19
20 OLLAMA_API = "http://localhost:11434/api/chat"
21 HEADERS = {"Content-Type": "application/json"}
22 MODEL_GPT = 'gpt-4o-mini'
23 MODEL_LLAMA = 'llama3.2'
24
25 ###
26 # Configura el entorno
27
28 load_dotenv()
29 api_key = os.getenv('OPENAI_API_KEY')
30
31 if api_key and api_key[:8]=='sk-proj-':
32     print("La clave de API parece buena")
33 else:
34     print("¿Puede haber un problema con tu clave API
35     ? ¡Visita el cuaderno de resolución de problemas!")
36
37 openai = OpenAI()
```

```

37 ###
38 # Aquí está la pregunta; escribe sobre esto para
   preguntar algo nuevo
39
40 system_prompt = "Necesito que actues como un
   programador experto en Python. \
41 Responde lo más detallado posible, dando ejemplos. \
42 Responde en formato markdown.\n"
43
44 user_prompt = """
45 Explica qué hace este código y por qué:
46 yield from {book.get("author") for book in books if
   book.get("author")}
47 """


---


48 ###
49 # Haz que gpt-4o-mini responda, con streaming
50
51 def python_expert():
52     stream = openai.chat.completions.create(
53         model=MODEL_GPT,
54         messages=[
55             {"role": "system", "content":
system_prompt},
56             {"role": "user", "content": user_prompt}
57         ],
58         stream=True
59     )
60
61     response = ""
62     display_handle = display(Markdown(""), display_id
= True)
63
64     for chunk in stream:
65         delta = chunk.choices[0].delta
66         content = getattr(delta, 'content', '')
67         if content:
68             response += content
69             update_display(Markdown(response),
display_id=display_handle.display_id)
70
71     return response

```

```
72
73
74
75 # Llamada a la función
76 final_response = python_expert()
77
78
79 #%%
80 # Haz que Llama 3.2 responda
81
82 messages = [
83     {
84         "role": "user",
85         "content": (
86             "Necesito que actúes como un programador
87             experto en Python. "
88             "Responde lo más detallado posible,
89             dando ejemplos. "
90             "Responde en formato markdown. "
91             "Explica qué hace este código y por qué
92             : "
93             "yield from {book.get('author') for book
94             in books if book.get('author')}]"
95         )
96     }
97 ]
98
99 payload = {
100     "model": MODEL_LLAMA,
101     "messages": messages,
102     "stream": False
103 }
104
105 response = requests.post(OLLAMA_API, json=payload,
106     headers=HEADERS)
107
108 # Procesar la respuesta JSON
109 response_json = response.json()
110
111 # Extraer el contenido de la respuesta
112 message_content = response_json.get('message', {}).
113
```

```
107 get('content', 'No content received')
108
109 # Mostrar en formato Markdown
110 display(Markdown(message_content))
```